



Solemne 1

INDICACIONES

- Esta Solemne se realiza de forma individual.
- La solemne debe ser rendida en la plataforma <http://codetrack.cl/>
- Para entrar debe escribir su correo institucional y como password su rut (solo numeros).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Clasificación de estrellas por luminosidad.

Enunciado

La luminosidad relativa de una estrella de masa M (en masas solares) se estima mediante la relación:

$$L = M^{3,5}$$

- Escriba un programa que:
 - Solicite al usuario la masa M de una estrella.
 - Implemente una función `luminosidad(M)` que calcule L .
 - Utilice un ciclo `while` para procesar múltiples valores de masa hasta que el usuario ingrese `-1`.
 - Utilice una estructura `if/else` para indicar si la estrella es más o menos luminosa que el Sol.

Solución.

Idea

El problema consiste en calcular la luminosidad de una estrella a partir de su masa usando la relación $L = M^{3,5}$.

Se requiere:

- Definir una función que reciba la masa M y retorne la luminosidad L .
- Pedir valores al usuario repetidamente usando un ciclo `while`.
- Terminar el ciclo cuando el usuario ingrese `-1`.
- Comparar la luminosidad obtenida con la del Sol ($L = 1$) para clasificar la estrella.

Implementación

```
1 # Función que calcula la luminosidad
2 def luminosidad(M):
3     return M**3.5
4
5 # Bucle principal
6 while True:
7     # Solicitar masa
8     M = float(input("Ingrese la masa de la estrella (-1 para salir): "))
9
10    # Condición de término
11    if M == -1:
12        print("Programa finalizado.")
13        break
14
15    # Calcular luminosidad
16    L = luminosidad(M)
17
18    # Clasificación
19    if L > 1:
20        print(f"Luminosidad = {L:.2f} → Más luminosa que el Sol")
21    else:
22        print(f"Luminosidad = {L:.2f} → Menos luminosa que el Sol")
```

Resultado

El programa permite ingresar múltiples masas de estrellas, calcular su luminosidad y clasificarlas en relación con el Sol, utilizando funciones, ciclos y estructuras condicionales.

2. Caída libre y conservación de la energía mecánica.

Enunciado

Considere una partícula de masa m que cae verticalmente bajo la acción de la gravedad, despreciando la resistencia del aire. Suponga que la velocidad inicial es nula, $v_0 = 0$, y que la aceleración es constante, $g = 9,8 \frac{m}{s^2}$. La posición y la velocidad de la partícula se describen en función del tiempo t como:

$$y(t) = y_0 - \frac{1}{2}gt^2, \quad v(t) = -gt$$

donde y_0 es la altura inicial ingresada por el usuario.

La energía cinética, la energía potencial gravitatoria y la energía mecánica total se definen como:

$$E_c = \frac{1}{2}mv(t)^2, \quad E_p = mgy(t), \quad E_{\text{tot}} = E_c + E_p$$

■ Escriba un programa que:

- Defina una función `caida_libre(m, g=9.8)`.
- Solicite al usuario el valor de la altura inicial y_0 .
- Use un ciclo `for` para recorrer tiempos enteros entre $t = 1$ y $t = 99$.
- Calcule en cada instante la posición y y la velocidad v usando las expresiones de caída libre.
- Use esas mismas variables y y v para calcular la energía potencial E_p y la energía cinética E_c , respectivamente. En todo instante de tiempo.
- Calcule la energía total como la suma de ambas energías.
- Muestre en pantalla, para cada instante, los valores de E_p , E_c y E_{tot} .

Solución.

Idea

Se debe construir una función que modele la caída libre de una partícula de masa m . La función pedirá la altura inicial y_0 y luego recorrerá distintos instantes de tiempo. En cada iteración se calcula primero la posición y y la velocidad v . Después, esas mismas cantidades se reutilizan en las fórmulas de energía:

$$E_c = \frac{1}{2}mv^2, \quad E_p = mgy$$

Finalmente, la energía mecánica total se obtiene sumando ambas:

$$E_{\text{tot}} = E_c + E_p$$

Implementación

```
1 def caida_libre(m, g = 9.8):
2     y0 = float(input('ingresar valor'))
3
4     for t in range(1, 100):
5         y = y0 - 1/2 * g * t**2
6         v = -g * t
7
8         e_c = 1 / 2 * m * v**2
9         e_p = m * g * y
10        e_tot = e_c + e_p
11
12        print('=' * 20)
13        print(f' para E_p {e_p} y E_k {e_c}: Energia total = {e_tot:.2f}')
```

Comentario

En este programa, la variable y calculada con la ecuación de posición se usa directamente en la energía potencial e_p , mientras que la variable v calculada con la ecuación de velocidad se usa directamente en la energía cinética e_c . De este modo, el código refleja de manera explícita la dependencia física entre movimiento y energía.

3. Aproximación de $\sin(x)$ mediante serie de Taylor.

Enunciado

La función seno $\sin(x)$ se puede aproximar mediante su serie de Taylor:

$$\sin(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

- Escriba un programa que:

- Solicite al usuario un valor real x .
- Solicite un entero positivo n (número de términos).
- Calcule la suma de los primeros n términos de la serie usando un ciclo `for`.
- Compare el resultado con `math.sin(x)`.
- Puede usar la función `factorial()` del módulo `math`.

- Bonus:

- Puede crear su propia función que implemente el factorial para obtener un bonus de décimas.

Solución.

Idea

La serie de Taylor de $\sin(x)$ es una suma alternante de términos impares. Cada término tiene la forma:

$$(-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

Para aproximar la función:

- Se suman los primeros n términos de la serie.
- Se utiliza un ciclo `for` para iterar sobre los valores de k .
- Se compara el resultado con el valor real usando `math.sin(x)`.

Implementación

```
1 import math
2
3 # Solicitar datos al usuario
4 x = float(input("Ingrese el valor de x: "))
5 n = int(input("Ingrese el número de términos: "))
6
7 # Inicializar suma
8 suma = 0
9
10 # Cálculo de la serie
11 for k in range(n):
12     termino = ((-1)*k) * (x**(2*k + 1)) / math.factorial(2*k + 1)
13     suma += termino
14
15 # Resultado aproximado
16 print(f"Aproximación de sin(x): {suma}")
17
18 # Valor real
19 valor_real = math.sin(x)
20 print(f"Valor real de sin(x): {valor_real}")
21
22 # Error absoluto
23 error = abs(valor_real - suma)
24 print(f>Error absoluto: {error}")
```

Resultado

El programa aproxima $\sin(x)$ mediante una serie de Taylor y permite comparar la precisión de la aproximación en función del número de términos utilizados.